DHBW Heilbronn

# Software Engineering II.1 – Block 6

## Whitebox, Wartung und Change Request

Code verstehen, sicher ändern – und eine Änderung des Auftraggebers kontrolliert umsetzen

Software Engineering II.1 | WS 2026/27 | Dr. rer. nat. Ana Nicolaescu

# Wo stehen wir im Kurs?

| Block 1 | Block 2 | Block 3 | Block 4 | Block 5 | Block 6 | Block 7 |
|---|---|---|---|---|---|---|
| 4 UE | 4 UE | 4 UE | 8 UE | | 4 UE | 4 UE |
| Einführung & Vorgehensmodelle | Requirements Engineering | Entwurf & Entwurfsmuster | Implementierung & Versionsmanagement | Testen (Blackbox) | Whitebox, Wartung & Change Request | Management & Abschlusspräsentation |

**Durchgängiges Gruppenprojekt „Photo Album“: jeder Block liefert ein prüfbares Artefakt ins Repository**

| | | | | | | |
|---|---|---|---|---|---|---|
| Teams, Repo, Vorgehen | REQUIREMENTS.md Use Cases · AK · Prototyp | ARCHITECTURE.md Klassen- · Sequenzdiagramm | Build · Git · PR Review · CI grün | TESTING.md Unit/Integr. · Blackbox | Coverage · Refactoring Change Request | RISK_ANALYSIS.md Präsentation |

# Einstieg: Ist dieser Code getestet?

Einstieg

**Eine Methode aus dem Photo Album und der einzige Test dazu. Der Test ist grün.**

**3 min**

Plenum

**METHODE**

```
// Klasse PhotoLabel
String titleOf(Photo photo) {
    String t = null;              // ①
    if (photo.hasTitle()) {       // ②
        t = photo.title();        // ③
    }
    return t.toUpperCase();       // ④
}
```

**TEST T1**

```
@Test
void titleIsUppercase() {
    Photo p = new Photo("strand.jpg", "Sommer");
    assertEquals("SOMMER", label.titleOf(p));
}   // grün
```

**FRAGEN**

1. Gehen Sie T1 Zeile für Zeile durch: Welche der Anweisungen ① bis ④ werden ausgeführt?
2. Ist titleOf damit vollständig getestet?
3. Finden Sie eine Eingabe, bei der titleOf abstürzt.

**ERGEBNIS** T1 führt alle vier Anweisungen aus – und trotzdem steckt ein Fehler in der Methode.

# Der Kontrollflussgraph von titleOf

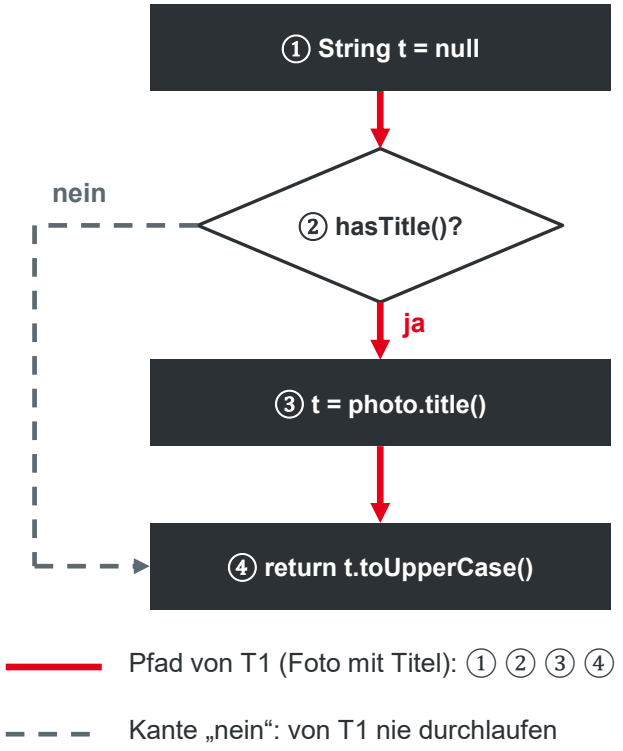
```
// Klasse PhotoLabel
String titleOf(Photo photo) {
    String t = null;              // ①
    if (photo.hasTitle()) {       // ②
        t = photo.title();        // ③
    }
    return t.toUpperCase();       // ④
}
```

**Knoten** = eine Anweisung (①, ③, ④) oder eine Entscheidung (② als Raute).

**Kante** = ein möglicher Weg von einer Anweisung zur nächsten.

**Eine Entscheidung hat zwei Ausgänge:** ja und nein. Das if ohne else hat einen versteckten Ausgang: „nein“ springt direkt zu ④.

**Ein Testlauf** = ein Pfad durch den Graphen.




**T1 besucht jeden Knoten, aber nicht jede Kante. Der Fehler liegt auf der Kante, die kein Test benutzt.**

# Statement Coverage und Branch Coverage am Beispiel

### Statement Coverage (Anweisungsüberdeckung, C0)

**ausgeführte Anweisungen ÷ alle Anweisungen**

Frage: Wurde jede Anweisung mindestens einmal ausgeführt? Hier: 4 Anweisungen (① bis ④).

### Branch Coverage (Zweigüberdeckung, C1)

**durchlaufene Ausgänge ÷ alle Ausgänge aller Entscheidungen**

Frage: Wurde jede Entscheidung mindestens einmal mit ja UND mit nein durchlaufen? Hier: 1 Entscheidung (②) = 2 Ausgänge.

| Test | Eingabe | Pfad | Statement | Branch | Fehler gefunden? |
|---|---|---|---|---|---|
| **T1** | Foto mit Titel „Sommer“ | ① ② ③ ④ | 4/4 = 100 % | 1/2 = 50 % | nein |
| **T2** | Foto ohne Titel | ① ② ④ | 3/4 = 75 % | 1/2 = 50 % | ja: NullPointerException |
| **T1 + T2** | beide Tests zusammen | alle Kanten | 4/4 = 100 % | 2/2 = 100 % | ja |

**T1 allein: Statement 100 %, Branch nur 50 %. Erst die Branch-Lücke zeigt, dass der Fall „ohne Titel“ fehlt.**

# Wie hängen Statement und Branch Coverage zusammen?

**100 % Branch ⇒ 100 % Statement (gilt immer)**

Jede Anweisung liegt hinter einem Ausgang einer Entscheidung (oder am Anfang). Wer alle Ausgänge nimmt, kommt an jeder Anweisung vorbei.

**100 % Statement ⇏ 100 % Branch**

Gegenbeispiel T1: Bei einem if ohne else besucht der ja-Ausgang alle Anweisungen. Der nein-Ausgang enthält keine Anweisung – Statement Coverage bemerkt ihn nicht.

**So zeigt es JaCoCo (Maven: target/site/jacoco)**

**Lines / Instructions** = Statement Coverage
**Branches** = Branch Coverage

**grün:** Zeile ausgeführt, alle Zweige durchlaufen

**gelb:** Zeile ausgeführt, aber nicht alle Zweige – so sieht Zeile ② nach T1 aus

**rot:** Zeile nie ausgeführt

**Branch Coverage ist das stärkere Kriterium. Für die Domänenklassen messen und interpretieren Sie deshalb Branch Coverage.**

# Übung: Coverage für canAdd berechnen

Übung

## Gleiches Vorgehen, zweite Methode: Darf ein Foto noch in das Album?

5 min

```
// Klasse Album
boolean canAdd(Photo photo) {
    if (photos.size() >= 100) {      // ①
        return false;                // ②
    }
    if (photos.contains(photo)) {    // ③
        return false;                // ④
    }
    return true;                     // ⑤
}
```

**Test T1:** Album mit 10 Fotos, ein neues Foto.
Erwartet: canAdd liefert true.

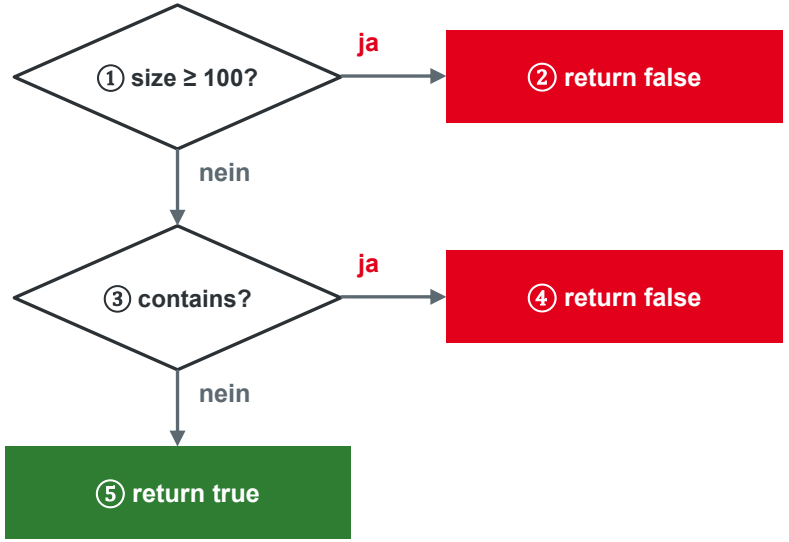
**AUFTRAG**

1. Zeichnen Sie den Kontrollflussgraphen (Knoten ① bis ⑤, Rauten für ① und ③).
2. Welchen Pfad läuft T1? Berechnen Sie Statement und Branch Coverage von T1.
3. Welche Tests brauchen Sie zusätzlich für 100 % Branch Coverage? Geben Sie die Eingaben an.

**ERGEBNIS** Graph-Skizze, zwei Prozentwerte für T1, Liste der fehlenden Tests.

**DANACH** Lösung auf der nächsten Folie gemeinsam vergleichen.

# Lösung: canAdd




| Test | Eingabe | Pfad | Statement | Branch |
|---|---|---|---|---|
| **T1** | 10 Fotos, neues Foto | ① ③ ⑤ | 3/5 = 60 % | 2/4 = 50 % |
| **T2** | genau 100 Fotos | ① ② | 2/5 = 40 % | 1/4 = 25 % |
| **T3** | 10 Fotos, Foto schon drin | ① ③ ④ | 3/5 = 60 % | 2/4 = 50 % |
| **T1–T3** | alle drei zusammen | alle | 5/5 = 100 % | 4/4 = 100 % |

**Faustregel:** Mindestzahl Tests für 100 % Branch = Anzahl Entscheidungen + 1. Hier: 2 + 1 = 3.

Diese Zahl heißt **zyklomatische Komplexität V(G)** (McCabe). Je höher, desto mehr Tests – und desto schwerer zu ändern.

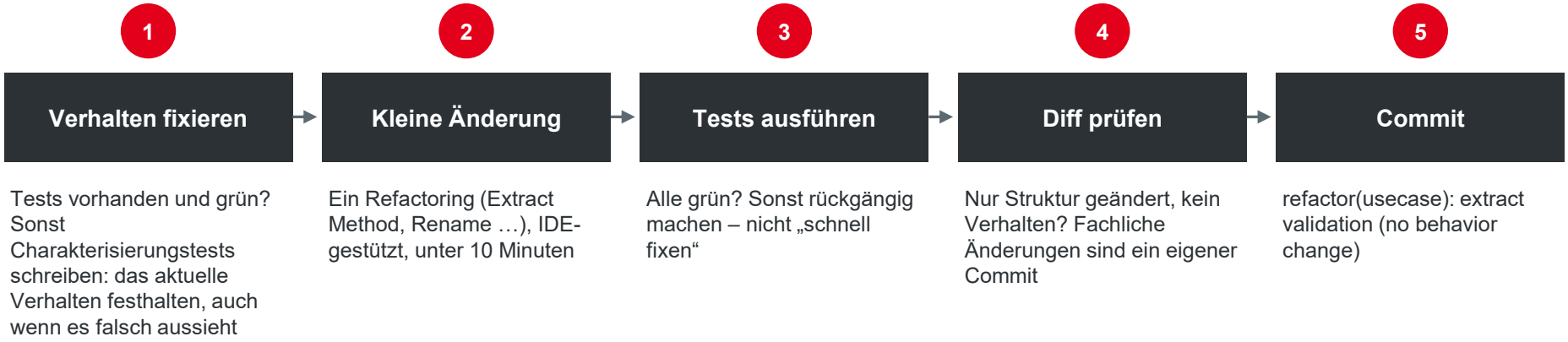
**Branch Coverage sagt, welcher Zweig fehlt. Die Grenzwertanalyse (Block 5) sagt, mit welchem Wert: genau 100 Fotos.**

# Vier Arten der Wartung (ISO/IEC 14764)

| Art | Auslöser | Beispiel im Photo Album | Absicherung |
|---|---|---|---|
| **Korrektiv** | Fehler im Betrieb | Hash über den Pfad statt den Inhalt: Duplikate werden nicht erkannt | Regressionstest, der den Fehler reproduziert, bleibt |
| **Adaptiv** | Umgebung ändert sich | Neue Java-Version; neues Bildformat (HEIC) | gesamte Suite in CI; Kompatibilitätstests |
| **Perfektiv** | neue oder bessere Funktion | Suche nach Titel; schnellere Albumansicht; Change Request | neue Tests plus bestehende als Regression |
| **Präventiv** | Risiko senken, bevor etwas bricht | God Class zerlegen; Abhängigkeit aktualisieren | Verhalten vorher mit Tests fixieren, dann ändern |

**In der Praxis: perfektiv etwa 50 %, adaptiv 25 %, korrektiv 20 %, präventiv 5 %. Die Sorte, die niemand einplant, führt zu technischen Schulden.**

# Refactoring in sicheren Schritten




**Legacy-Code ohne Tests?**

Reverse Engineering = verstehen, ohne zu ändern (Call Hierarchy, Abhängigkeitsgraph, Tests als Dokumentation). Reengineering = gezielt verbessern, ohne das Verhalten zu ändern: erst Verhalten mit Tests festhalten, dann Nahtstellen schaffen, dann Schritt für Schritt umbauen. Ein Big-Bang-Rewrite ist fast immer teurer.

**Refactoring = Struktur ändern, Verhalten erhalten. Wer beides gleichzeitig tut, weiß bei einem roten Test nicht, was schuld ist.**

# Change Request CR-1: Fotos in mehreren Alben und Tags

**CHANGE REQUEST CR-1 · Auftraggeber · Priorität: hoch · Ziel: Release 1.1.0 zu Block 7**

| | |
|---|---|
| **CR-1.1** | Ein Foto muss gleichzeitig in mehreren Alben enthalten sein können. Die Datei wird dabei nur einmal gespeichert. Titel und Beschreibung gehören zum Foto, nicht zum Album. |
| **CR-1.2** | Das Entfernen eines Fotos aus einem Album darf das Foto in anderen Alben nicht verändern. Ein Foto, das in keinem Album mehr liegt, muss nach Bestätigung endgültig gelöscht werden können. |
| **CR-1.3** | Die Nutzerin muss einem Foto bis zu 5 Tags (je 1–20 Zeichen, eindeutig je Foto, ohne Unterschied bei Groß-/Kleinschreibung) zuweisen und entfernen können. |
| **CR-1.4** | Die Nutzerin muss die Fotos eines Albums nach einem Tag filtern können; die Albumansicht zeigt die Anzahl der Treffer. |

**Unverändert:** max. 100 Fotos je Album (ein Foto zählt in jedem Album, in dem es liegt); Duplikatregel gilt je Album; alle bisherigen Akzeptanzkriterien bleiben gültig.

**Auftrag heute: Impact Analysis (CHANGE_REQUEST.md) – nicht programmieren. Umsetzung bis Block 7 mit Tag v1.1.0.**

# Mini-Impact-Analysis

Teamarbeit

**Bevor jemand Code ändert: Was trifft die Änderung? Antworten Sie mit Namen und IDs aus Ihrem Projekt.**

**4 min**

**AUFTRAG**

1. Welche Anforderung (ID aus REQUIREMENTS.md) widerspricht CR-1?
2. Welche Klasse ändert sich am stärksten? (Tipp: Wo steht heute die Liste der Fotos?)
3. Welcher bestehende Test wird nach der Änderung rot? Nennen Sie den Testnamen.

**ERGEBNIS** Drei Antworten als erste Stichpunkte in CHANGE_REQUEST.md.

**DANACH IM PLENUM** Zwei Teams nennen ihre Antwort auf Frage 3. Der Rest ist Hausaufgabe.

# Bis zum nächsten Block

## Hausaufgabe (Team)

- TESTING.md: JaCoCo-Report öffnen (./mvnw verify → target/site/jacoco/index.html). Statement- und Branch-Werte je Schicht notieren, gelbe Zeilen in der Domäne erklären, mindestens 2 Tests für fehlende Zweige ergänzen
- CHANGE_REQUEST.md fertigstellen: betroffene Anforderungen, Klassen, Tests (mit IDs), Risiken, Aufwand, Plan
- Change Request umsetzen, Regressionstests grün, Release v1.1.0
- REFACTORING.md: ein Refactoring nach den fünf Schritten, mit Begründung
- Storyboard der Abschlusspräsentation (max. 8 Folien)

## Merken Sie sich

- Statement Coverage: Wurde jede Anweisung ausgeführt?
- Branch Coverage: Wurde jede Entscheidung mit ja und nein durchlaufen?
- 100 % Branch ⇒ 100 % Statement, nicht umgekehrt
- Mindestzahl Tests für 100 % Branch: Entscheidungen + 1

**Tag „block-6“ nach Abschluss der Hausaufgabe setzen.**

# Quellen und Materialbasis Block 6

- Spillner & Linz: Basiswissen Softwaretest; ISTQB Foundation Level (Whitebox-Verfahren, Überdeckungen)
- McCabe (1976): A Complexity Measure; Rapps & Weyuker (1985): Selecting Software Test Data Using Data Flow Information
- ISO/IEC 14764 (Software-Wartung); Lientz & Swanson (1980): Software Maintenance Management
- Fowler: Refactoring (2. Aufl.); Cunningham (1992): technische Schulden; Fowler: Strangler Fig Application
- Feathers (2004): Working Effectively with Legacy Code; Chikofsky & Cross (1990): Reverse Engineering
- JaCoCo-Dokumentation (Coverage Counters); Vorwissen: Block 5 (Blackbox, Mutation), Block 3 (Schichten, Pattern-Gericht)